AI VIET NAM
@aivietnam.edu.vn

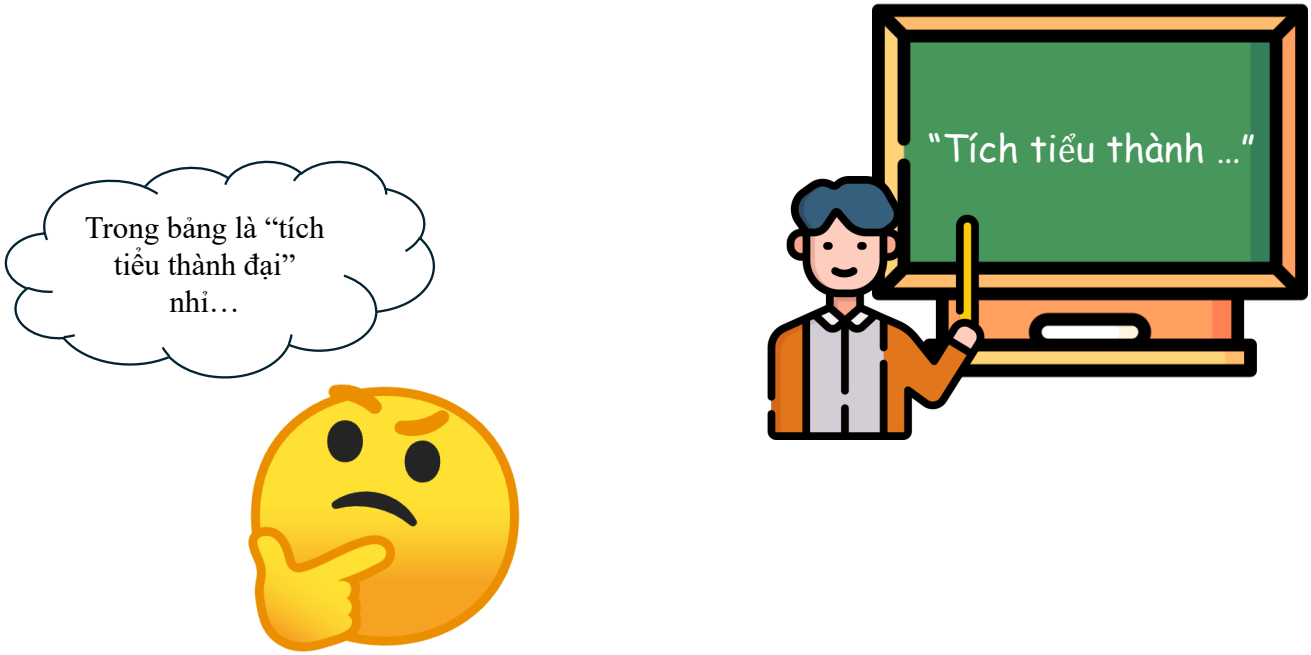
AI VIET NAM – AI COURSE 2025

LLMs Series: Pre-training GPT

Dương Trường Bình Tô Phát Đạt Dương Đình Thắng Đinh Quang Vinh

I. Dẫn nhập

Hãy tưởng tượng chúng ta đang chơi trò "điền từ còn thiếu vào câu". Nếu ta có câu: "Tích tiểu thành ...", có lẽ hầu hết chúng ta sẽ trả lời được ngay là "đại" phải không? Rõ ràng là vậy, nhưng, tại sao chúng ta lại có thể làm được cũng như phản xạ nhanh đến thế nhỉ?



Hình 1: Minh họa về trò chơi điền vào ô trống.

Thật ra, không có gì huyền bí cả. Đó là vì có lẽ, từ lúc nhỏ đến khi lớn, chúng ta đã nghe hoặc đọc cụm từ này hàng trăm lần. Bộ não chúng ta đã tự động ghi nhớ một quy luật: nếu có cụm từ "Tích tiểu thành" thì xác suất cực kỳ cao từ tiếp theo sẽ là "đại". Cơ chế này diễn ra một cách tự nhiên đến mức chúng ta không cần phải suy nghĩ hay phân tích ngữ pháp gì cả. Đơn giản là chúng ta đang thực hiện một việc duy nhất: *dự đoán từ tiếp theo.*

Để tiện theo dõi, các bạn có thể tham khảo bảng các thuật ngữ được dùng trong bài tại Mục VII..

Mục lục

I. Dẫn nhập **1**

II. Mô hình ngôn ngữ là gì? **3**

III. Sự chuyển mình: từ mô hình ngôn ngữ đến LLM **5**

IV. Generative Pre-trained Transformer **7**

IV.1. Pre-training 7

IV.2. Fine-tuning 9

V. Tổng kết **10**

VI. Câu hỏi trắc nghiệm **11**

VII. Tài liệu tham khảo **16**

Bảng thuật ngữ **17**

Phụ lục **18**

Nếu để ý kỹ hơn, các bạn sẽ thấy kiểu "dự đoán từ tiếp theo" này thực ra cực kỳ quen thuộc trên chiếc điện thoại hay máy tính mà chúng ta dùng hằng ngày. Chẳng hạn, khi các bạn vừa gõ chữ "Thời tiết" vào ô tìm kiếm của Google, nó sẽ tự động hiện ra gợi ý là "hôm nay" hoặc "ngày mai". Hay khi nhắn tin, bàn phím điện thoại cũng hay tự đề xuất trước cho chúng ta những từ mà nó đoán là chúng ta định viết tiếp.

Hình 2: Chức năng dự đoán từ tiếp theo trên Google tìm kiếm.

Quan sát ví dụ về Google tại Hình 2, các bạn có thể thấy rõ cơ chế này đang âm thầm hoạt động để đưa ra các gợi ý phù hợp nhất dựa trên những gì chúng ta đã gõ. Thực chất, mọi ứng dụng từ đơn giản như bàn phím điện thoại cho đến những thứ nâng cap mà người ta hay nhắc tới gần đây, về bản chất đều có chung một "gốc rễ" vận hành: nhìn vào những gì chúng ta vừa gõ để đưa ra phán đoán về cái gì sẽ xuất hiện kế tiếp.

II. Mô hình ngôn ngữ là gì?

Nếu nhìn từ góc độ máy tính như các ví dụ trên, câu hỏi đặt ra là làm thế nào để chúng ta hiện thực hóa được một tính năng dự đoán như vậy? Để giải quyết bài toán này, cụ thể trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP), người ta đã sáng tạo và sử dụng một thuật toán với tên gọi

là *language model (mô hình ngôn ngữ)*. Đây cũng chính là thành phần cốt lõi đằng sau các ứng dụng chatbot mà chúng ta thấy hiện nay.

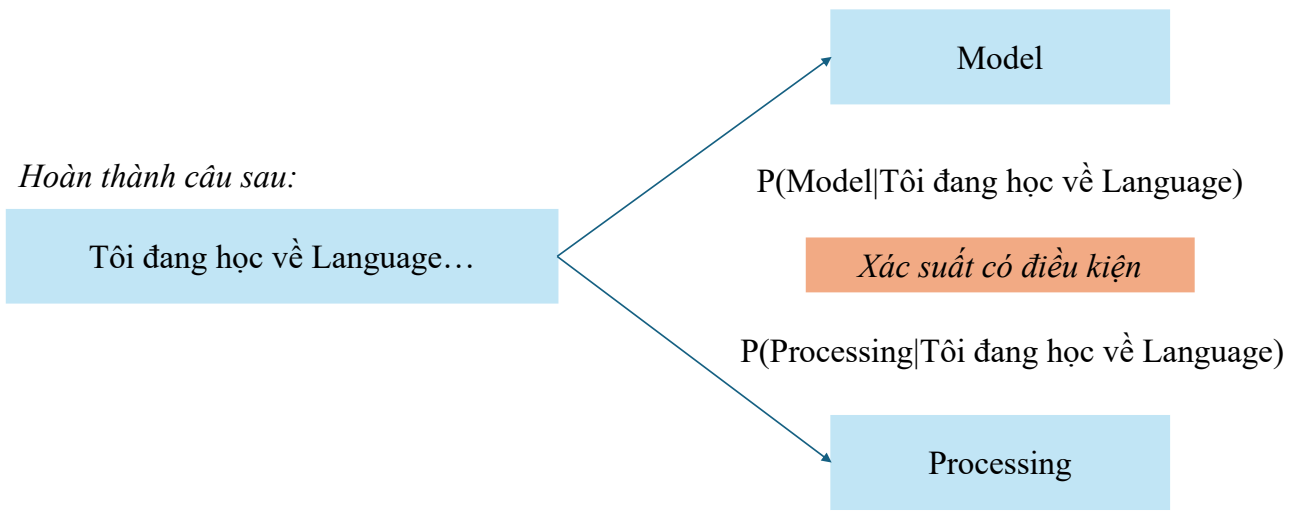
Định nghĩa toán học của language model

Về tổng quan, language model là một thuật toán được thiết kế để ước lượng phân phối xác suất của một chuỗi các từ ngữ. Nói một cách cụ thể, nhiệm vụ duy nhất của mô hình ngôn ngữ là tính toán từ tiếp theo nào có khả năng xuất hiện cao nhất dựa trên những gì đã được viết trước đó. Xét một chuỗi văn bản gồm $n-1$ từ đầu tiên là $x_1, x_2, \ldots, x_{n-1}$. Xác suất P để dự đoán từ tiếp theo x_n (tức từ thứ n) dựa trên ngữ cảnh trước đó được tính bằng xác suất có điều kiện (*conditional probability*):

$$P(x_n \mid x_1, x_2, \ldots, x_{n-1})$$

Mô hình sẽ chọn từ x_n sao cho giá trị xác suất này đạt mức cao nhất.

Để trực quan hóa khái niệm trên, chúng ta hãy cùng xem xét một ví dụ thực tế ngay dưới đây để thấy rõ hơn cách thức vận hành của mô hình:



Hình 3: Mô hình ngôn ngữ sử dụng xác suất có điều kiện để lựa chọn từ tiếp theo phù hợp nhất.

Ví dụ về dự đoán từ tiếp theo

Giả sử chúng ta đưa vào một ngữ cảnh là: *"Tôi đang học về Language..."*. Lúc này, mô hình sẽ tính toán xác suất cho các ứng cử viên tiềm năng ở vị trí tiếp theo. Nó có thể so sánh giữa hai từ "Model" và "Processing". Dựa trên kho dữ liệu nó từng đọc, nó nhận thấy xác suất $P(\text{Model} \mid \text{Tôi đang học về Language})$ lớn hơn hẳn so với $P(\text{Processing} \mid \text{Tôi đang học về Language})$. Do đó, từ "Model" sẽ được chọn trong trường hợp này.

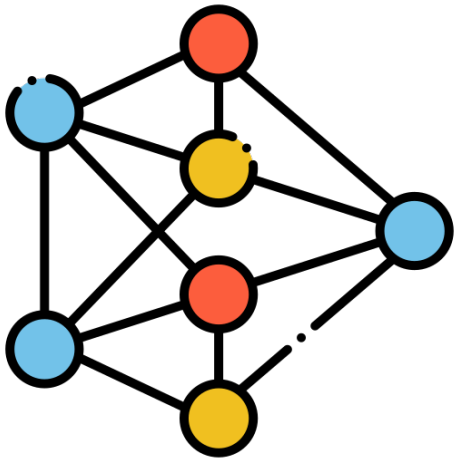
III. Sự chuyển mình: từ mô hình ngôn ngữ đến LLM

Với cách thức vận hành trên của mô hình ngôn ngữ, ta hoàn toàn có thể đưa vào một tập ngữ liệu vừa đủ để máy tính học được những nội dung, câu giao tiếp cơ bản hằng ngày. Tuy nhiên, một câu hỏi thú vị được đặt ra là: Nếu chúng ta muốn thuật toán này có thể làm được những việc phức tạp hơn, ví dụ như viết một bài luận, giải một bài toán hay thậm chí là lập trình, thì liệu "vốn liếng" của nó như vậy đã đủ chưa?

Câu trả lời là: Rất khó. Nếu vốn liếng của nó chỉ dừng lại ở vài cuốn sách giáo khoa. Một mô hình ngôn ngữ nhỏ giống như một đứa trẻ mới học đánh vần. Nó có thể đoán đúng các từ thông dụng, nhưng chưa đủ trải nghiệm để hiểu được những tầng nghĩa sâu xa hay kiến thức chuyên môn phức tạp. Để đi tìm giải pháp cho vấn đề trên với khái niệm về mô hình ngôn ngữ, các nhà nghiên cứu đã đặt ra một giả thuyết rằng nghe có vẻ đơn giản nhưng rất quan trọng là: *"Điều gì sẽ xảy ra nếu chúng ta cho mô hình đọc toàn bộ kho tàng tri thức của nhân loại?"*

Thay vì phải thay đổi thuật toán hay thiết kế lại các công thức phức tạp cho từng bài toán cụ thể, giới nghiên cứu nhận thấy rằng chúng ta chỉ cần giữ nguyên nguyên lý dự đoán từ tiếp theo, nhưng thực hiện việc "nâng cấp" quy mô một cách đồng bộ:

Mở rộng về số lượng dữ liệu



Mở rộng về số lượng tham số mô hình

Hình 4: Dữ liệu và tham số mô hình - hai yếu tố quan trọng của một mô hình ngôn ngữ lớn.

Như minh họa tại Hình 4, việc mở rộng quy mô này tập trung vào hai thành phần chính yếu:

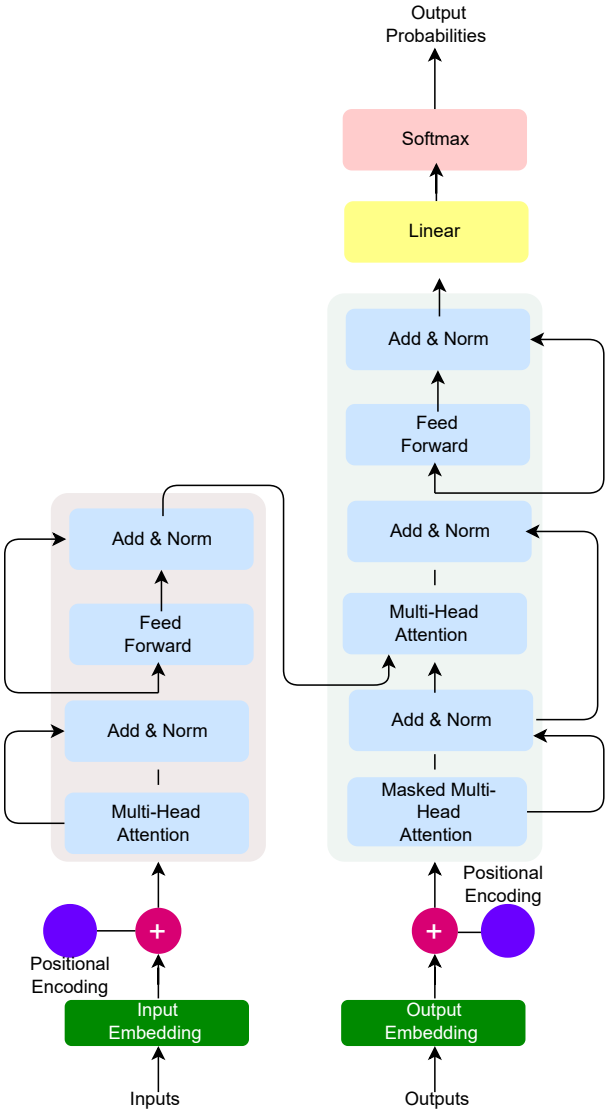
- **Mở rộng về dữ liệu:** Thay vì vài nghìn câu, chúng ta cho mô hình tiếp cận với hàng tỷ trang văn bản từ Internet, sách vở và tri thức nhân loại.

- **Mở rộng về kiến trúc (Tham số):** Chúng ta xây dựng một "bộ não" mô hình khổng lồ với hàng tỷ tham số (*parameters*) để có đủ sức chứa và ghi nhớ các quy luật phức tạp từ lượng dữ liệu bao la kia.

Khi sự kết hợp giữa lượng dữ liệu lớn và số lượng tham số cực đại đạt đến một ngưỡng nhất định, một hiện tượng thú vị sẽ xảy ra: mô hình không chỉ dừng lại ở việc có khả năng dự đoán từ đơn thuần tốt hơn, mà bắt đầu xuất hiện những khả năng mới như suy luận logic, trả lời câu hỏi hay phân loại văn bản. Đây chính là tiền đề ra đời của khái niệm *large language model (LLM)*, hay còn gọi là *mô hình ngôn ngữ lớn*. Đây cũng chính là động lực phát triển của mô hình GPT, một trong những công trình nghiên cứu quan trọng và là nền tảng cho sự ra đời của ChatGPT sau này.

IV. Generative Pre-trained Transformer

GPT (Generative Pre-trained Transformer) [1] là mô hình ngôn ngữ lớn mang tính bước ngoặt được công bố bởi OpenAI vào năm 2018. Trong khi nhiều hệ thống đương thời sử dụng các kiến trúc phức tạp và được tinh chỉnh hẹp cho từng tác vụ riêng biệt, GPT tiên phong sử dụng kiến trúc **decoder-only Transformer** [2] (minh họa bởi Hình 5) làm nền tảng. Nghĩa là, nó hoàn toàn loại bỏ phần encoder, chỉ tập trung vào sức mạnh của cơ chế *masked self-attention* để dự đoán từ tiếp theo một cách tuần tự (autoregressive). Mục tiêu cốt lõi của công trình nghiên cứu này là cải thiện khả năng hiểu ngôn ngữ của mô hình thông qua việc tận dụng nguồn dữ liệu văn bản khổng lồ không dán nhãn.

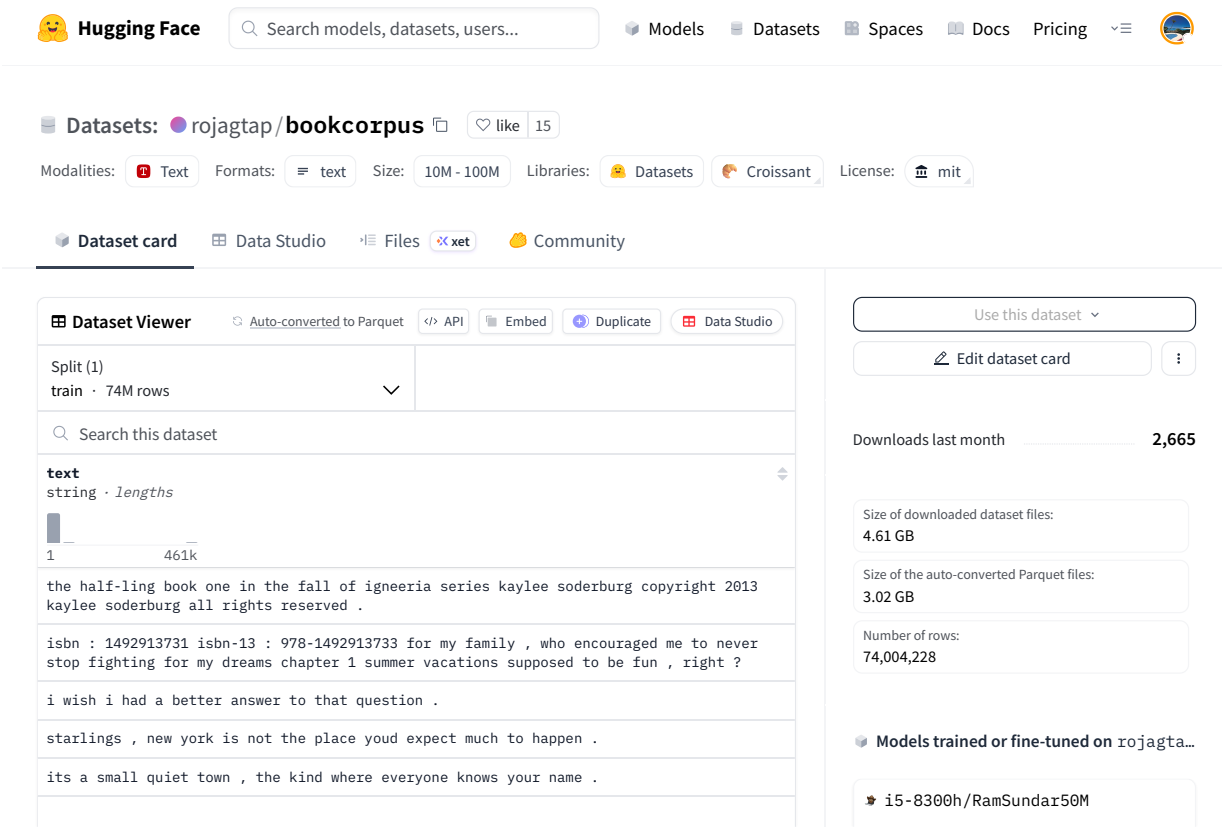


Hình 5: Minh họa kiến trúc Transformer gốc. GPT chỉ sử dụng khối decoder bên phải.

IV.1. Pre-training

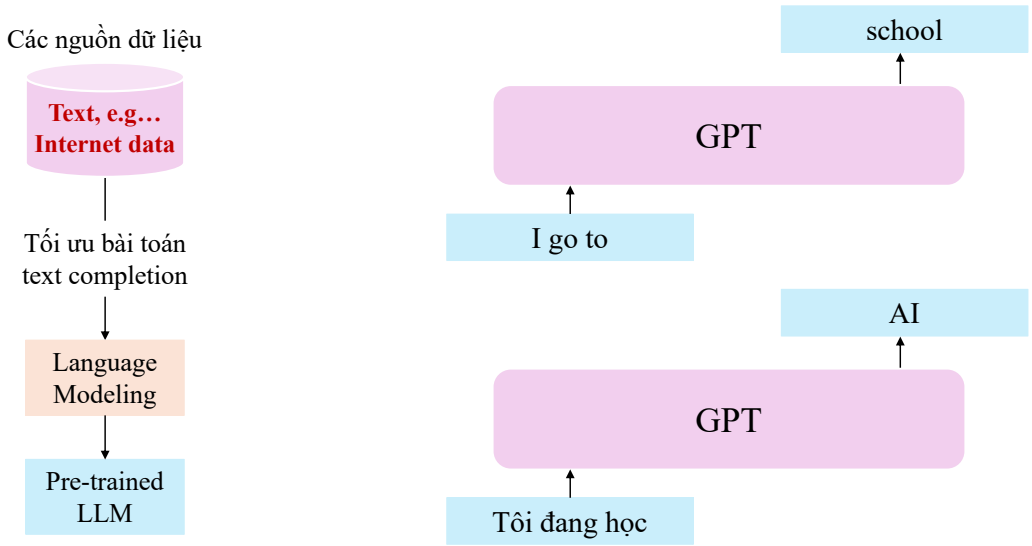
Thay vì thiết kế các kiến trúc riêng biệt cho từng tác vụ phức tạp, GPT đề xuất một framework gồm hai giai đoạn: tiền huấn luyện (*generative pre-training*) và tinh chỉnh (*discriminative fine-tuning*). Ở bước đầu tiên, mô hình cần xây dựng "vốn liếng" ngôn ngữ thông qua việc tự học trên lượng dữ liệu đủ lớn và chất lượng.

Đầu tiên, để xây dựng một "vốn liếng" ngôn ngữ phong phú, bước đầu tiên là cho mô hình tiếp cận với một lượng dữ liệu đủ lớn và chất lượng.



Hình 6: Tập dữ liệu BooksCorpus trên HuggingFace được sử dụng để huấn luyện GPT.

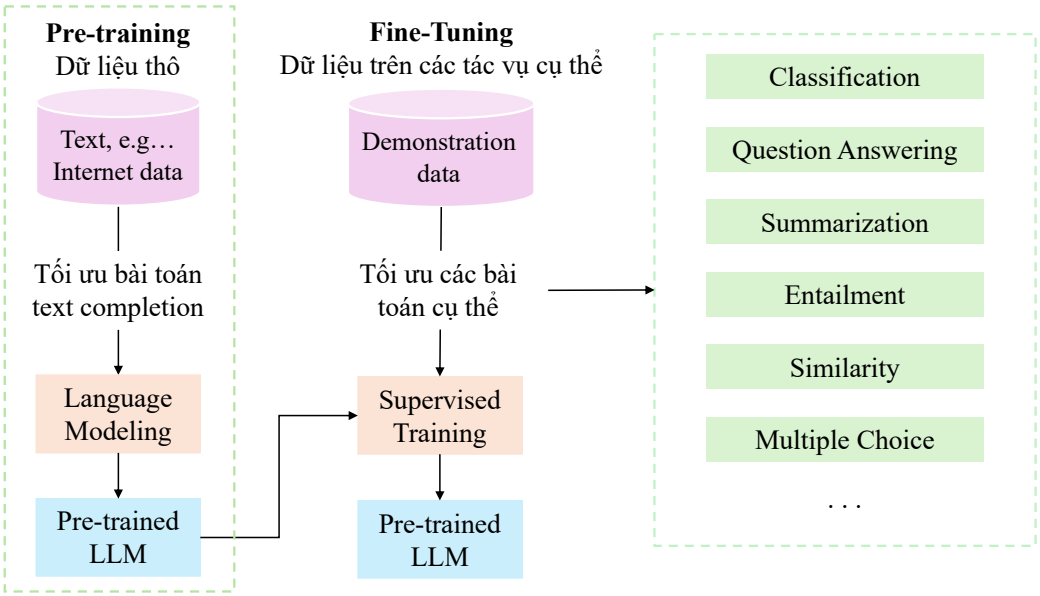
Như minh họa tại Hình 6, mô hình được huấn luyện trên tập dữ liệu *BooksCorpus* chứa hơn 7,000 cuốn sách chưa xuất bản thuộc nhiều thể loại khác nhau, tạo nên một bộ dữ liệu huấn luyện tới hơn 74 triệu mẫu, kích thước xấp xỉ 4.61GB. Việc tiếp cận với lượng dữ liệu khổng lồ và đa dạng này cho phép mô hình tích lũy một lượng kiến thức thế giới phong phú. Từ nguồn dữ liệu bao la này, GPT sẽ thực hiện nhiệm vụ cốt lõi là tối ưu hóa việc dự đoán từ tiếp theo, hay ở giai đoạn này còn được gọi là *pretraining*.



Hình 7: Quy trình từ dữ liệu thô đến mô hình pre-trained LLM thông qua bài toán text completion.

Dựa trên sơ đồ tại hình 7, quy trình này được thực hiện bằng cách tối ưu hóa bài toán hoàn thiện văn bản (*text completion*). Thông qua việc liên tục dự đoán từ tiếp theo dựa trên ngữ cảnh—như các ví dụ "I go to → school" hay "Tôi đang học → AI"—mô hình dần nắm bắt được các quy luật liên kết từ ngữ phức tạp.

IV.2. Fine-tuning



Hình 8: Quy trình từ dữ liệu thô đến mô hình pre-trained LLM và ứng dụng vào các tác vụ hạ nguồn thông qua Fine-tuning.

Theo kỳ vọng, một mô hình sau khi trải qua giai đoạn pre-training để nắm bắt các quy luật ngôn ngữ tổng quát sẽ đóng vai trò là một nền tảng vững chắc (*Pre-trained LLM*). Như minh họa tại Hình 8, thay vì phải xây dựng lại từ đầu, chúng ta có thể tận dụng mô hình đã có này và thực hiện bước *tinh chỉnh (fine-tuning)* với một lượng nhỏ *dữ liệu đặc thù được gán nhãn (demonstration data)*. Quá trình này giúp mô hình thích nghi nhanh chóng để giải quyết hiệu quả *các tác vụ cụ thể hơn (downstream tasks)* đa dạng như *phân loại (classification)*, *hỏi đáp (question answering)*, hay *tóm tắt văn bản (summarization)*.

Kết quả thực tế từ nghiên cứu của OpenAI [1] cho thấy chiến lược "học rộng trước, tinh chỉnh sau" này mang lại hiệu quả vượt trội so với việc thiết kế mô hình biệt lập cho từng bài toán đơn lẻ. Cụ thể, GPT đã vượt qua các phương pháp dán nhãn truyền thống trên 9 trong số 12 tập dữ liệu thử nghiệm. Những bước tiến đáng kể nhất được ghi nhận trong khả năng suy luận logic (*commonsense reasoning*) với mức tăng 8.9% và kỹ năng trả lời câu hỏi (*question answering*) tăng 5.7% so với các công nghệ trước đó.

V. Tổng kết

Tóm lại, thông qua hành trình từ một thuật toán dự đoán từ đơn giản đến sự ra đời của GPT, chúng ta đã phần nào thấy được tiềm năng của các mô hình ngôn ngữ lớn (LLM). Sức mạnh của LLM không chỉ nằm ở việc dự đoán từ tiếp theo một cách máy móc, mà còn ở khả năng tích lũy tri thức thế giới từ nguồn dữ liệu thô khổng lồ để giải quyết các bài toán phức tạp như suy luận logic hay trả lời câu hỏi một cách tự nhiên.

Hy vọng bài viết này sẽ mang lại cho bạn đọc một cái nhìn tổng quan, xúc tích và thực tế về cách thức vận hành cũng như tiềm năng phát triển của các mô hình ngôn ngữ lớn hiện nay.

VI. Câu hỏi trắc nghiệm

Câu 1. Các mô hình thuộc họ GPT (Generative Pre-trained Transformer) có điểm gì khác biệt về cấu trúc so với kiến trúc Transformer được giới thiệu trong bài báo "Attention Is All You Need"?

A. GPT chỉ sử dụng khối Encoder kết hợp với bidirectional attention để mã hóa toàn bộ ngữ cảnh trước khi sinh token đầu ra.

B. GPT vẫn giữ đầy đủ Encoder và Decoder, nhưng đảo thứ tự giữa feed-forward network và multi-head attention để tối ưu hóa tính song song.

C. GPT thay thế self-attention bằng cross-attention để liên kết trực tiếp input sequence với output sequence mà không cần hidden states trung gian.

D. GPT chỉ sử dụng khối Decoder với masked self-attention để dự đoán token tiếp theo từ các token quá khứ, và trong thiết lập chuẩn không cần khối Encoder riêng.

Câu 2. Trong causal language modeling, cách ghép input và target nào sau đây đúng nhất cho mục tiêu *next-token prediction*?

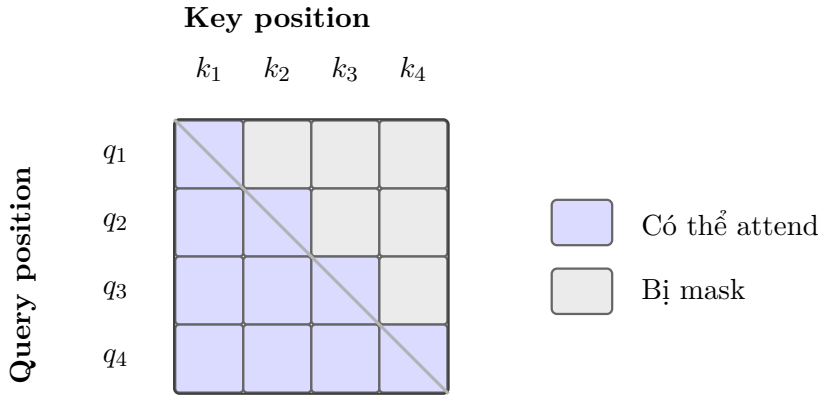
A. Input là $[x_1, x_2, x_3, x_4]$, target là $[x_1, x_2, x_3, x_4]$ vì mô hình cần tái tạo lại chính xác toàn bộ chuỗi đầu vào.

B. Input là [<bos>, x_1, x_2, x_3], target là $[x_1, x_2, x_3, x_4]$ vì ở mỗi vị trí mô hình học dự đoán token kế tiếp.

C. Input là $[x_1, x_2, x_3, x_4]$, target là một nhãn duy nhất y đại diện cho toàn chuỗi vì mục tiêu chính là sequence classification.

D. Input là [x_2, x_3, x_4, <eos>], target là $[x_1, x_2, x_3, x_4]$ vì mô hình cần học suy luận ngược từ tương lai về quá khứ.

Câu 3. Quan sát *causal attention mask* dưới đây của một decoder-only Transformer cho chuỗi gồm 4 vị trí. Nhận định nào sau đây đúng nhất về quá trình *training* của causal language model?



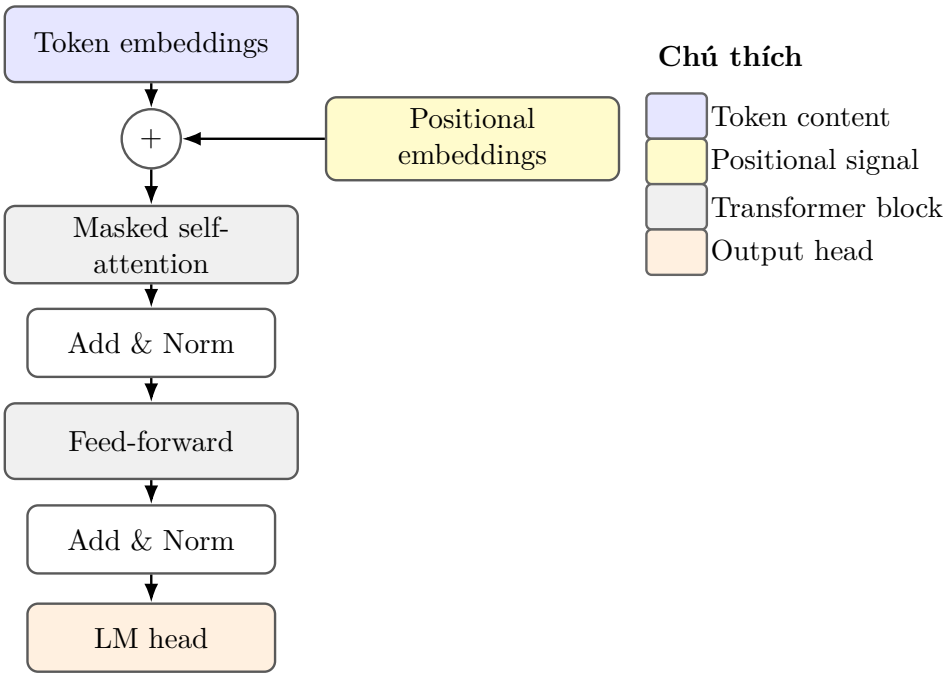
A. Vì có mask, mô hình buộc phải xử lý tuần tự từng vị trí như RNN, nên không thể tính loss cho nhiều vị trí trong cùng một forward pass.

B. Trong training, mô hình vẫn có thể tính song song hidden states, logits và loss cho nhiều vị trí; điều kiện là tại vị trí t, attention chỉ được truy cập các token ở vị trí $\leq t$.

C. Causal mask chỉ chặn direct attention, nhưng qua residual connections mô hình vẫn nhận được thông tin từ các token tương lai, nên ràng buộc nhân quả chỉ đúng ở layer đầu tiên.

D. Attention mask chỉ cần dùng ở inference; trong training có thể bỏ đi vì target đã được dịch trái một vị trí nên causal property vẫn tự được bảo toàn.

Câu 4. Xét một block decoder-only Transformer dùng causal self-attention như hình dưới. Nếu bỏ hoàn toàn *positional embeddings* và không thay bằng bất kỳ *positional signal* nào khác, hệ quả nào sau đây đúng nhất?



A. Mô hình sẽ suy giảm mạnh khả năng biểu diễn thứ tự token, vì self-attention theo nội dung không tự mã hóa vị trí; causal mask chỉ cung cấp một tín hiệu vị trí rất hạn chế.

B. Mô hình tự động trở thành bidirectional language model vì khi không còn positional embeddings thì ràng buộc trái-sang-phải cũng biến mất.

C. Mô hình sẽ chuyển từ token-level modeling sang character-level modeling vì positional embeddings vốn quyết định đơn vị biểu diễn.

D. Ảnh hưởng chủ yếu chỉ là tốc độ hội tụ chậm hơn đôi chút, còn khả năng mô hình hóa thứ tự về cơ bản không đổi.

Câu 5. Khi triển khai xây dựng mô hình nanoGPT bằng PyTorch để huấn luyện trên một tập dữ liệu nhỏ, hàm mục tiêu cơ bản nhất mà mô hình tính toán để tối ưu hóa trong quá trình cập nhật trọng số là gì?

A. Cực đại hóa điểm số BLEU và ROUGE giữa văn bản do mô hình sinh ra tự do và văn bản gốc trong tập dữ liệu.

B. Cực tiểu hóa sai số Mean Squared Error giữa vector embedding của câu đầu vào và câu đầu ra.

C. Cực tiểu hóa hàm mất mát Cross-Entropy giữa phân bố xác suất dự đoán token tiếp theo do mô hình xuất ra và token thực tế trong chuỗi.

D. Cực tiểu hóa hàm Hinge Loss nhằm phân loại chính xác văn bản đầu vào thuộc phong cách Shakespeare hay thuộc các phong cách văn học khác.

Câu 6. Trong quá trình sinh văn bản, tham số temperature được áp dụng lên logits trước khi qua softmax. Khi đặt temperature rất thấp, gần tiến về 0, hành vi của mô hình thay đổi như thế nào?

A. Mô hình trở nên quyết đoán hơn, thường chọn token có xác suất cao nhất, nên đầu ra ổn định hơn nhưng kém đa dạng hơn.

B. Phân phối xác suất trở nên phẳng hơn, khiến mô hình dễ chọn các token hiếm và sáng tạo hơn.

C. Mô hình tự động loại bỏ mọi token có xác suất thấp hơn một ngưỡng cố định trước khi lấy mẫu.

D. Mô hình chuyển từ sinh tuần tự sang sinh song song trên toàn bộ chuỗi để tăng tốc suy luận.

Câu 7. Khi một mô hình GPT được scale lên rất lớn về mặt tham số và pretrain trên tập dữ liệu khổng lồ, nhận định nào sau đây là đúng nhất về năng lực nổi bật thường xuất hiện so với các mô hình nhỏ hơn?

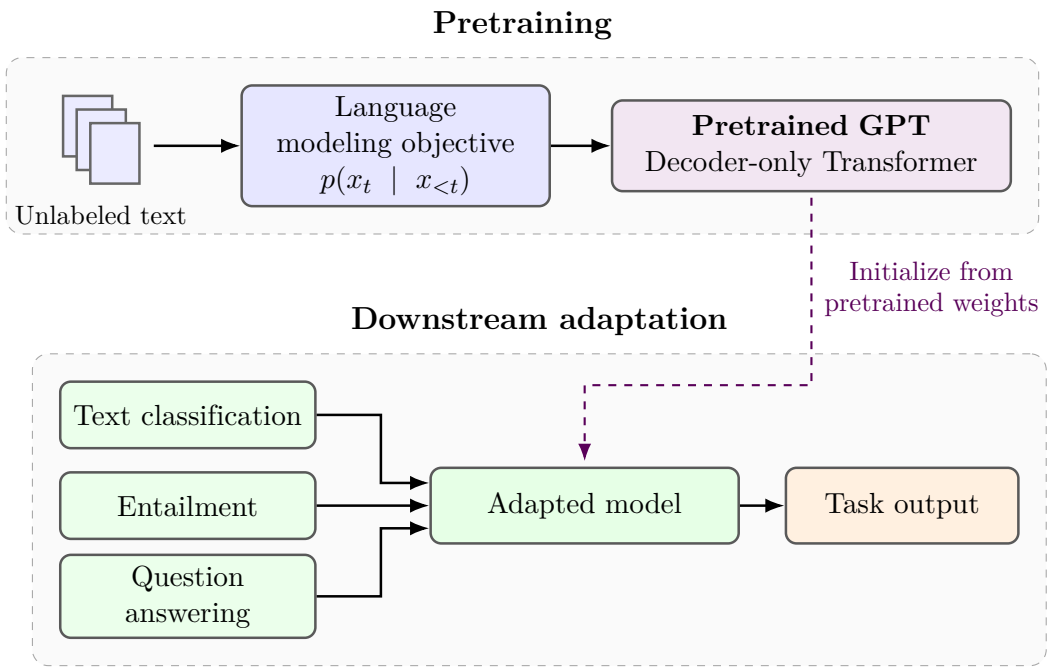
A. Mô hình có khả năng suy luận vượt trội, dẫn đến hình thành tri thức tổng quát. Từ đó sinh ra các dữ kiện luôn luôn chính xác.

B. Mô hình tự động có khả năng truy xuất dữ liệu thời gian thực từ các hệ thống bên ngoài mà không cần công cụ hỗ trợ.

C. Mô hình dần thể hiện in-context learning, tức là có thể làm một task mới tốt hơn chỉ từ ví dụ hoặc chỉ dẫn trong prompt mà không cần cập nhật trọng số.

D. Mô hình có thể tự chỉnh sửa lại trọng số của chính nó trong lúc suy luận, nên học tri thức mới ngay lập tức như con người.

Câu 8. Quan sát sơ đồ khái quát dưới đây về cách một mô hình GPT đi từ giai đoạn pre-training sang downstream adaptation. Phát biểu nào sau đây mô tả đúng nhất cơ chế chuyển tri thức này?



A. Với mỗi downstream task, GPT thay toàn bộ decoder backbone bằng một kiến trúc mới, vì một decoder-only model không đủ linh hoạt để thích nghi với nhiều định dạng input khác nhau.

B. GPT chuyển tri thức bằng cách pretrain một decoder chung trên unlabeled text, sau đó đưa input của từng task về dạng token sequence tương thích, gắn một task-specific head gọn nhẹ, và fine-tune mô hình đã pretrain.

C. GPT chủ yếu chuyển tri thức bằng cách đóng băng hoàn toàn pretrained decoder như một feature extractor cố định, rồi chỉ huấn luyện lớp output mới cho từng task.

D. GPT chỉ dùng được cho text classification; còn các task như entailment hay question answering thì về bản chất bắt buộc phải đổi sang encoder-decoder trước khi fine-tune.

Câu 9. Giả sử bạn có cùng *compute budget* và cùng số lượng token huấn luyện. Bạn phải chọn giữa hai corpus để pretrain một causal language model. Lựa chọn nào sau đây hợp lý nhất?

A. Chọn corpus web thô, nhiều nhiễu và chứa nhiều near-duplicate, vì chỉ cần đủ nhiều token thì mô hình sẽ tự học cách loại bỏ phần lớn noise.

B. Chọn corpus đã được filter và deduplicate tốt, vì khi compute hữu hạn, chất lượng và mật độ thông tin của token ảnh hưởng trực tiếp đến chất lượng representations mà mô hình học được.

C. Hai corpus sẽ cho chất lượng gần như tương đương, vì next-token prediction chỉ quan tâm đến số lượng token chứ hầu như không nhạy với chất lượng dữ liệu.

D. Chọn corpus có càng nhiều đoạn lặp lại càng tốt, vì lặp lại nội dung gần giống nhau luôn cải thiện khả năng khái quát hóa của mô hình.

Câu 10. Hallucination là hiện tượng mô hình sinh ra thông tin sai với độ tự tin cao. Đâu là cách giải thích đúng nhất cho hiện tượng này ở các mô hình GPT?

A. Do temperature và top-p mặc định luôn ở mức rất cao, nên mô hình bị buộc phải ưu tiên token hiếm để duy trì độ đa dạng.

B. Do context window hữu hạn gây tràn bộ nhớ attention, khiến toàn bộ hidden states bị biến dạng trong mọi truy vấn dài.

C. Do hallucination chỉ xuất hiện khi câu hỏi nằm ngoài hoàn toàn phân bố dữ liệu huấn luyện, nên với kiến thức phổ biến mô hình sẽ không bịa đặt.

D. Do mục tiêu tối ưu cốt lõi của mô hình là dự đoán token tiếp theo theo phân phối đã học, chứ không phải đối chiếu sự thật như một hệ truy xuất hay cơ sở tri thức có kiểm chứng.

VII. Tài liệu tham khảo

[1] A. Radford et al., "Improving language understanding by generative pre-training", OpenAI, Tech. Rep., 2018. [Online]. Available: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

[2] A. Vaswani et al., "Attention is all you need", in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762.

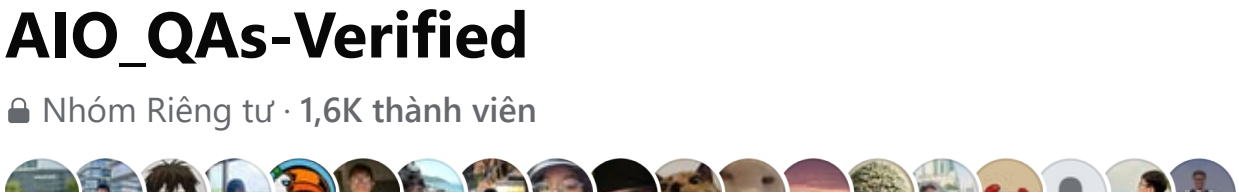
[3] Y. Zhu et al., "Aligning books and movies: Towards story-like visual explanations by watching movies and reading books", in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 19–27. [Online]. Available: https://arxiv.org/abs/1506.06724.

Bảng thuật ngữ

Thuật ngữ	Tạm dịch	Mô tả
Fine-tuning	Tinh chỉnh	Giai đoạn huấn luyện tiếp theo sau pre-training, sử dụng tập dữ liệu nhỏ có dán nhãn để cập nhật trọng số, giúp mô hình thích nghi và giải quyết tốt một tác vụ cụ thể.
GPT	Generative Pre-trained Transformer	Một họ mô hình ngôn ngữ do OpenAI phát triển, tiên phong sử dụng kiến trúc decoder-only Transformer kết hợp với phương pháp học tiền huấn luyện (pre-training) trên lượng lớn dữ liệu không dán nhãn.
Language Model	Mô hình ngôn ngữ	Thuật toán được thiết kế để ước lượng phân phối xác suất của một chuỗi từ ngữ, thực hiện nhiệm vụ dự đoán từ tiếp theo có khả năng xuất hiện cao nhất dựa trên ngữ cảnh đã cho.
LLM	Mô hình ngôn ngữ lớn	Large language model (LLM), mô hình trí tuệ nhân tạo được huấn luyện trên kho ngữ liệu khổng lồ nhằm học các quy luật ngôn ngữ và khả năng sinh văn bản tự nhiên một cách mạch lạc.
Parameters	Tham số	Các biến số bên trong mạng nơ-ron của mô hình được tối ưu hóa liên tục trong quá trình huấn luyện, đóng vai trò lưu trữ tri thức học được từ tập dữ liệu.
Pre-training	Tiền huấn luyện	Giai đoạn đầu tiên và cốt lõi trong huấn luyện LLM, mô hình tự học quy luật ngôn ngữ, cú pháp và kiến thức thế giới từ tập dữ liệu khổng lồ thông qua bài toán next-token prediction.
Transformer	Kiến trúc Transformer	Kiến trúc mạng nơ-ron cốt lõi trong các LLM hiện đại, sử dụng cơ chế self-attention để bắt các liên kết ngữ cảnh phức tạp mà không cần xử lý tuần tự toàn bộ chuỗi đầu vào như RNN.

Phụ lục

1. **Datasets:** Các file dataset phục vụ cho nội dung bài này có thể được tải tại đây.
2. **Hint:** Các file code gợi ý có thể được tải tại đây.
3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.



Hình 9: Hình ảnh group facebook AIO Q&A.